INDIVIDUAL PROJECT REPORT

ON

“CampusConnect: Student Academic & Communication Portal”

Submitted in partial fulfillment of the requirement for the award of Degree of
Bachelor of Technology in Computer Science & Engineering

Submitted to:

Rajasthan Technical University, Kota (Raj.)

Submitted by:

AKSHAT VERMA
Roll No.: 24EARCS015
Responsibility: Clubs & Events Module

Under the guidance of

PROJECT GUIDE
Er. Ram Babu Buri
Assistant Professor, Dept. of CSE

UNDER THE GUIDANCE OF
Er. Ram Babu Buri
Assistant Professor, Dept. of CSE

Session: 2026–27

Department of Computer Science & Engineering

ARYA COLLEGE OF ENGINEERING & I.T.

SP-42, RIICO Industrial Area, Kukas, Jaipur, Rajasthan 302028

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

CERTIFICATE

This is to certify that the individual project report entitled “CampusConnect: Student Academic & Communication Portal — Clubs & Events Module” has been submitted by Akshat Verma (Enrollment / Roll No.: 24EARCS015), a student of Bachelor of Technology in Computer Science & Engineering at Arya College of Engineering & I.T., Jaipur, affiliated with Rajasthan Technical University, Kota, during the academic session 2026–27.

This report accurately records the student’s personal contribution as an active individual member of the five-person engineering team that developed the CampusConnect portal. The technical work documented herein is strictly confined to the Clubs & Events Subsystem, including requirements analysis, domain modeling, REST API design, Java/Spring Boot business services, React user interface components, and cross-module integration with shared identity and dashboard services. The work was carried out under my direct supervision and satisfies the academic standards prescribed by the University.

Er. Ram Babu Buri
Project Guide
Assistant Professor, Dept. of CSE
Arya College of Engg. & I.T., Jaipur

Signature: _____
Date: _____

Dr. Akhil Pandey
Head of Department
Professor & HOD, Dept. of CSE
Arya College of Engg. & I.T., Jaipur

Signature: _____
Date: _____

Place: Jaipur, Rajasthan

DECLARATION

I, **Akshat Verma**, Roll No. **24EARCS015**, student of Bachelor of Technology in Computer Science & Engineering at Arya College of Engineering & I.T., Jaipur, hereby declare that this individual project report entitled **"CampusConnect: Student Academic & Communication Portal — Clubs & Events Module"** is a bona fide record of my own personal engineering work completed during the development period from **6 July 2026 to 6 October 2026** under the guidance of **Er. Ram Babu Buri**.

- **Individual Responsibility:** My primary development responsibility was exclusively the **Clubs & Events Module**, comprising full-stack design, Spring Boot controllers and services, JPA persistence, MySQL schema definitions, React views, and dashboard event integration.
- **Boundary of Authorship:** I do not claim sole authorship of the broader CampusConnect system. The foundational modules—Authentication & User Profile (Aman Kumar Chauhan), Academic & Assignments (Aditya Gour), Announcements & Notifications (Anjali Kumari), and Chat & Community (Ayushman Pathak)—were implemented in parallel by my teammates and are referenced only to explain technical interfaces.
- **Empirical Verification:** All architectural patterns, REST API routes, entity models, database constraints, Git commit metrics (branch akshat-feature, commit 77c357d and merge 244fae8), and build verifications presented in this report reflect actual verified code within the project repository.
- **Academic Integrity:** No features, automated test suites, deployment claims, or performance metrics have been fabricated. Where items remain pending or were verified manually rather than via automated unit tests, they are explicitly designated as such.

Date: _____
Place: Jaipur, Rajasthan

Akshat Verma
Enrollment No.: 24EARCS015
B.Tech. CSE, Session 2026–27
Signature: _____

ACKNOWLEDGEMENT

I express my deepest gratitude and sincere respect to my project guide, **Er. Ram Babu Buri**, Assistant Professor, Department of Computer Science & Engineering, for his steadfast guidance, constructive technical scrutiny, and invaluable advice. His emphasis on robust relational modeling, strict service-layer authorization boundaries, and systematic Git branch isolation played a pivotal role in shaping my approach to the Clubs & Events module.

I am deeply thankful to **Dr. Akhil Pandey**, Head of the Department of Computer Science & Engineering, for providing the advanced computing laboratories, departmental infrastructure, and academic ecosystem that facilitated continuous development and collaborative software engineering.

I would like to convey my sincere appreciation to my esteemed teammates—**Aman Kumar Chauhan** (Authentication & User Profiles), **Aditya Gour** (Academics, Assignments & Dashboard Layout), **Anjali Kumari** (Announcements & Notice Boards), and **Ayushman Pathak** (Real-Time Chat & Campus Community)—for their exceptional teamwork, mutual coordination, and discipline in maintaining shared configuration contracts and preventing upstream merge conflicts.

Finally, I extend my heartfelt gratitude to my family and friends for their unending encouragement, moral support, and patience throughout the rigorous 90-day development cycle of this project.

Akshat Verma
Roll No.: 24EARCS015
B.Tech. Computer Science & Engineering

ABSTRACT

CampusConnect is an enterprise-grade Student Academic and Communication Portal engineered by a five-member undergraduate team at Arya College of Engineering & I.T., Jaipur. The platform unifies disparate collegiate workflows—including academic subject management, institutional announcements, real-time messaging, user identity, and student extracurriculars—into a cohesive, web-based software ecosystem. This individual project report comprehensively documents the engineering lifecycle, design decisions, and empirical source code contributions of **Akshat Verma (24EARCS015)**, who served as the lead developer for the **Clubs & Events Subsystem**.

Prior to this implementation, extracurricular club discovery and campus event scheduling relied on fragmented social media channels, paper circulars, and manual roster sheets, leading to poor student awareness, rampant duplicate registrations, and administrative friction. The Clubs & Events module resolves these challenges through a secure, full-stack digital solution. The backend architecture is built with **Java 23** and **Spring Boot 3**, utilizing **Spring Data JPA**, **Hibernate ORM**, and **MySQL 8** (InnoDB) for persistent relational storage. The client application is developed with **React 18**, **Vite**, and **Tailwind CSS**, communicating via RESTful JSON APIs managed by **Axios**.

Key capabilities personally implemented and verified include: (i) multi-criteria club catalog discovery with category categorization and text search; (ii) an idempotent membership lifecycle supporting join, leave, and reactivation states protected by composite unique constraints; (iii) event scheduling with strict temporal validation (preventing registration deadlines that follow event inception); (iv) deterministic event registration enforcing hard seat capacity, pre-event deadlines, and cancellation tracking; (v) multi-tiered Role-Based Access Control (RBAC) allowing students to participate while restricting club creation to administrators and event coordination to designated faculty/student coordinators; and (vi) cross-module event aggregation inside the shared student/admin dashboard. Development was executed on the isolated akshat-feature Git branch between **6 July 2026 and 6 October 2026**, yielding **37 changed files and 3,804 lines of verified code**, confirmed through full Maven and Vite compilation.

Keywords: CampusConnect, Clubs & Events Subsystem, Spring Boot 3, Java 23, React 18, Vite, REST API, Hibernate, MySQL, Role-Based Access Control (RBAC), Event Registration Lifecycle, Idempotent Membership.

TABLE OF CONTENTS

Chapter 1 — Introduction **9**
1.1 Project Background 9
1.2 Problem Statement 9
1.3 Purpose of CampusConnect 9
1.4 Purpose of Clubs & Events Module 9
1.5 Objectives of Individual Contribution 9
1.6 Ninety-Day Development Period 10
1.7 Method and Report Organization 10
Chapter 2 — Project Overview, Scope and Responsibilities **11**
2.1 Five-Member Engineering Team Structure 11
2.2 Overall Architecture & Module Division 11
2.3 Individual Ownership: Clubs & Events 11
2.4 In-Scope vs. Out-of-Scope Work 12
2.5 Cross-Module Dependencies & Shared Conventions 12
2.6 Individual Contribution Matrix 12
Chapter 3 — Technology Stack and Development Environment **13**
3.1 Multi-Tier Technology Stack 13
3.2 Development & Build Environment (Java 23 / Node 20) 13
3.3 Engineering Conventions & Architectural Discipline 13
3.4 Local Workspace & Branch Setup 13
Chapter 4 — Requirements Analysis **14**
4.1 Domain Actors & Personas 14
4.2 Functional Requirements (FR-C1..C5, FR-E1..E6) 14
4.3 Non-Functional Requirements (Security, Integrity, Performance) 15
4.4 Business Rules & Domain Integrity Constraints 15
Chapter 5 — System Architecture and Design **16**
5.1 Layered System Architecture (Figure 5.1) 16
5.2 Clubs & Events Relational Data Model (Figure 5.2) 17
5.3 Backend Structural Pattern (Controller-Service-Repository) 17
5.4 Key Architectural Decisions & Integrity Strategy 18
5.5 Transaction Management & Concurrency Control 18
5.6 Data Security, Authorization Boundaries & Auditability 18
Chapter 6 — Clubs & Events Module: Design and Implementation **19**
6.1 Clubs Subsystem Architecture & Endpoints 19
6.2 Club Discovery, Category Filtering & Search 19
6.3 Idempotent Membership Lifecycle (Join/Leave/Reactivate) 19
6.4 Event Subsystem Architecture & Scheduling 19
6.5 Event Registration State Machine & Capacity Rules 20

6.6 Event Lifecycle & Sequence Modeling (Figures 6.1, 6.2, 6.3) 20
6.7 Role-Based Access Control Assertions 21
6.8 Centralized Business Validation & Exception Handling 21
6.9 Individual Authorship Summary for Clubs & Events 21
Chapter 7 — Frontend and User Interface Contributions 22
7.1 UI Architecture & Route Integration (/clubs) 22
7.2 Page Implementation: ClubsPage.jsx 22
7.3 Card & Modal Component Architecture (Figure 7.1) 22
7.4 Reactive State Management, Validation & Accessibility 23
Chapter 8 — API Integration and Backend Coordination 24
8.1 Shared Axios Client & JWT Interceptor 24
8.2 Complete Clubs & Events REST API Specification Table 24
8.3 Backend Coordination: ClubService & ClubEventService 25
8.4 Shared Dashboard Event Aggregation (dashboardService.js) 25
8.5 Network Resilience & Graceful Degradation Strategy 25
Chapter 9 — Git Workflow, Branching and Collaboration 26
9.1 Feature Branch Isolation (akshat-feature) 26
9.2 Verified Git Commit Evidence (77c357d & 244fae8) 26
9.3 Inter-Module Coordination & Branching Discipline 26
9.4 Shared-File Discipline & Conflict Prevention Protocol (Figure 9.1) 27
9.5 Pull Request Synchronization & Clean Working Tree Audit 27
9.6 Team Communication & Agile Cadence 27
Chapter 10 — Testing, Debugging and Verification 28
10.1 Quality & Verification Strategy (Figure 10.1) 28
10.2 Build Verification: Maven & Vite Results 28
10.3 Comprehensive Test Execution Table (TC-01..TC-15) 28
10.4 Debugging Log & Unit Test Audit Status 29
Chapter 11 — Implementation Results, Status and Limitations 30
11.1 End-to-End User Journey Walkthrough (Figure 11.1) 30
11.2 Feature Implementation Status Matrix 30
11.3 System Limitations & Boundaries 31
Chapter 12 — Challenges and Learning Outcomes 32
12.1 Technical Challenges & Engineering Solutions 32
12.2 Integration Challenges in a 5-Member Team 32
12.3 Key Technical & Professional Learnings 32
Chapter 13 — Future Scope and Conclusion 33
13.1 Planned Future Enhancements 33
13.2 Conclusion & Formal Individual Statement 33
References 34
Appendix A — Development Timeline and Verification Log 35

LIST OF FIGURES

Figure	Description	Page
Figure 5.1	Layered System Architecture of CampusConnect highlighting Clubs & Events Subsystem	**16**
Figure 5.2	Clubs & Events Relational Data Model (ER Diagram) & Relational Constraints	**17**
Figure 6.1	Clubs & Events Subsystem Use-Case Diagram across Student, Coordinator and Admin	**20**
Figure 6.2	Event Lifecycle & Registration State Machine showing Capacity & Deadline Constraints	**20**
Figure 6.3	Event Registration Sequence Diagram detailing Controller-Service-JPA Validation	**21**
Figure 7.1	Frontend Route, Component Hierarchy & Modal Dialog Architecture	**22**
Figure 9.1	Git Feature-Branching, Development Commits and Upstream Main Synchronization	**27**
Figure 10.1	Comprehensive Quality Assurance & Three-Pillar Verification Strategy	**28**
Figure 11.1	End-to-End User Journey from Dashboard Discovery to Verified Participation	**30**
Figure A.1	Ninety-Day Chronological Development Timeline (6 July 2026 – 6 October 2026)	**35**

LIST OF ABBREVIATIONS

Term	Expansion	Term	Expansion
API	Application Programming Interface	**JPA**	Jakarta Persistence API
CRUD	Create, Read, Update, Delete	**JSON**	JavaScript Object Notation
DTO	Data Transfer Object	**JWT**	JSON Web Token
ER	Entity–Relationship	**MVC**	Model–View–Controller
FK	Foreign Key	**NFR**	Non-Functional Requirement
FR	Functional Requirement	**ORM**	Object-Relational Mapping
HTTP	Hypertext Transfer Protocol	**PK**	Primary Key
IDE	Integrated Development Environment	**RBAC**	Role-Based Access Control
REST	Representational State Transfer	**SPA**	Single-Page Application
RTU	Rajasthan Technical University	**SQL**	Structured Query Language
UI/UX	User Interface / User Experience	**UK**	Unique Key / Constraint

CHAPTER 1 — INTRODUCTION

1.1 Project Background

Modern engineering institutions foster vibrant student ecosystems characterized by intense academic schedules, diverse departmental activities, and a rich culture of student-led organizations. At Arya College of Engineering & I.T., Jaipur, students engage in a multitude of technical societies, cultural associations, athletic leagues, and social initiative groups. However, in many collegiate environments, these activities suffer from extreme operational fragmentation. Academic notices exist on physical bulletin boards, lecture materials are distributed across ad-hoc cloud folders, and extracurricular announcements are disseminated through ephemeral messaging groups and printed posters. **CampusConnect: Student Academic & Communication Portal** was conceived as a unified web application to consolidate these disparate operational facets into an integrated, single-sign-on platform.

1.2 Problem Statement

Extracurricular clubs and campus events represent a critical pillar of collegiate life, fostering leadership, technical exploration, and collaborative learning. Despite their importance, student participation in clubs and events has historically faced acute logistical hurdles, specifically: (i) **Lack of Centralized Discovery:** Students frequently miss registration windows because announcements are scattered across unindexed chat channels; (ii) **Unregulated Registration:** Organizers lack reliable tools to enforce attendance ceilings, leading to overcrowded halls or abrupt on-the-spot cancellations; (iii) **Roster Inaccuracies:** Without database-backed membership tracking, club coordinators cannot verify active members or distribute targeted updates; and (iv) **Administrative Overhead:** Faculty and student coordinators spend excessive time compiling manual spreadsheets rather than focusing on event quality.

1.3 Purpose of CampusConnect

The overarching vision of CampusConnect is to establish a secure, multi-tenant digital campus for students, faculty members, and academic administrators. Engineered by a collaborative five-member undergraduate team, the portal delivers a responsive web dashboard incorporating authenticated role-based access, academic tracking, official institutional announcements, real-time community chat, and structured extracurricular management.

1.4 Purpose of My Clubs & Events Module

As an individual engineer within this five-member team, my specific mandate was to architect, implement, and integrate the **Clubs & Events Module**. The purpose of this module is to digitize the complete lifecycle of campus organizations and scheduled activities. It bridges the gap between campus organizers and the general student body by providing structured discovery mechanisms, transparent coordinator oversight, real-time membership rosters, and automated, rule-governed event registration.

1.5 Objectives of My Contribution

- Architect and implement a robust MySQL relational schema modeling clubs, memberships, events, and participant registrations.
- Develop RESTful Spring Boot controllers and business services enforcing strict authorization rules and business logic.
- Implement idempotent membership handling that prevents duplicate memberships and enables seamless join/leave transitions.
- Enforce strict event registration rules, ensuring that cancelled events reject applicants, registration deadlines are strictly upheld, and capacity caps are never breached.
- Design and construct intuitive React 18 frontend interfaces utilizing responsive Tailwind CSS cards, filter chips, and interactive modals.
- Integrate active club statistics and upcoming event feeds directly into the shared CampusConnect dashboard client.
- Maintain disciplined Git feature-branch isolation (akshat-feature) to ensure safe upstream synchronization without disrupting teammates.

1.6 Ninety-Day Development Period

The development lifecycle documented in this report spans exactly ninety days, from **6 July 2026 to 6 October 2026**. This timeframe encompassed initial requirements capture, architectural modeling, iterative backend and frontend implementation, cross-module contract alignment, rigorous manual validation, and final integration into the shared project codebase.

1.7 Method and Report Organization

This report follows an empirical, code-first engineering narrative. Chapter 2 outlines the team structure and scope boundaries. Chapter 3 documents the technology stack. Chapter 4 provides requirements analysis. Chapter 5 details system architecture and data models. Chapters 6, 7, and 8 provide in-depth explorations of the backend implementation, frontend components, and API contracts. Chapters 9, 10, and 11 present Git collaboration records, testing tables, and implementation status. Chapters 12 and 13 discuss engineering challenges, learning outcomes, future scope, and conclusions.

CHAPTER 2 — PROJECT OVERVIEW, SCOPE AND INDIVIDUAL RESPONSIBILITIES

2.1 Five-Member Engineering Team Structure

CampusConnect was engineered as a collaborative group capstone project by five B.Tech Computer Science & Engineering students at Arya College of Engineering & I.T. To ensure high cohesion and loose coupling, the portal was partitioned into five dedicated functional workstreams, with each team member assuming complete end-to-end responsibility for their respective module.

Member	Assigned Workstream	Scope & Core Responsibilities	Relationship to My Work
Aman Kumar Chauhan	**Authentication & User Profiles**	User entity, BCrypt hashing, JWT token issuance, Student/Teacher profiles, and Spring Security filter chain.	Provides authenticated Principal identity, User PK, and role context (STUDENT, TEACHER, ADMIN).
Aditya Gour	**Academics & Assignments**	Course subjects, assignment creation, student document submission, and shared responsive dashboard shell.	Defines common UI layout (MainLayout) and dashboard card widgets.
Anjali Kumari	**Announcements & Notifications**	Broadcast announcements, audience targeting, pinned bulletins, and institutional notice feeds.	Independent noticeboard module sharing common visual styling.
Ayushman Pathak	**Chat & Real-Time Communication**	WebSocket/STOMP private messaging, live presence tracking, and community feedback hub.	Independent real-time subsystem operating alongside REST services.
Akshat Verma (Self / Author)	**Clubs & Events Module**	**Full-stack design of Clubs, Memberships, Events, and Registrations, including RBAC, validation, and dashboard integration.**	**My complete individual workstream and personal engineering responsibility.**

2.2 Overall Architecture & Module Division

The architecture adheres to a clean separation of concerns. While each developer implemented domain-specific controllers, services, repositories, and UI views, all modules converge upon a unified React Single-Page Application (SPA) on the frontend and a centralized Spring Boot backend server interacting with a single MySQL relational database. Domain boundaries are strictly enforced at the package and table level.

2.3 Individual Ownership: Clubs & Events

I am Akshat Verma, and my sole development responsibility was the Clubs & Events Module. I designed and authored all database tables, JPA entities, repositories, service business rules, REST controllers, and React UI components associated with student clubs and campus events. I am not the author of authentication, academics, announcements, or chat, and I make no claim of ownership over those systems.

2.4 In-Scope vs. Out-of-Scope Work

Category	Technical Scope Boundaries
In-Scope Work (Personal Contribution)	• Design and migration of clubs, club_memberships, club_events, and event_registrations tables. • Spring Boot REST controllers: ClubController (/api/clubs) and ClubEventController (/api/events). • Business logic services: ClubService and ClubEventService with RBAC enforcement. • React UI views: ClubsPage.jsx, ClubCard.jsx, EventCard.jsx, and 4 modal dialogs. • Idempotent join/leave workflows and capacity-governed event registration. • Dynamic event and club metrics integration in dashboardService.js.
Out-of-Scope Work (Teammate Owned)	• JWT issuance, login/registration controllers, password encryption, and Security filter chain (Aman). • Course syllabus, assignment submission forms, and grading views (Aditya). • Institutional notice broadcasting, audience filtering, and pinned announcement algorithms (Anjali). • WebSocket broker setup, STOMP message endpoints, and community forum threads (Ayushman).

2.5 Cross-Module Dependencies & Shared Conventions

While maintaining rigorous code isolation, my module coordinates with the broader platform through standard contracts: (i) **Identity Contract:** Relies on the shared User entity and Spring Security Principal to extract the authenticated user's email without implementing custom session stores; (ii) **Role Model:** Reuses the institutional Role enumeration (STUDENT, TEACHER, ADMIN); (iii) **HTTP Client:** Reuses services/api.js configured with the global JWT Authorization interceptor; and (iv) **Dashboard Shell:** Employs MainLayout.jsx and registers the route /clubs inside App.jsx.

CHAPTER 3 — TECHNOLOGY STACK AND DEVELOPMENT ENVIRONMENT

3.1 Multi-Tier Technology Stack

To satisfy requirements of high performance, strong type safety, and modern responsive design, CampusConnect employs a contemporary full-stack architecture. Every technology in the stack was verified against the project repository files.

Layer	Technology & Version	Specific Role in Clubs & Events Module
Frontend Framework	React 18.3.1 (SPA)	Component-based user interface, hooks (useState, useEffect, useMemo, useCallback), conditional tab rendering.
Build Tool & Bundler	Vite 8.3.4	Fast Hot Module Replacement (HMR), optimized client packaging (built in 2.53s).
CSS & Styling	Tailwind CSS 3.4	Utility-first styling, responsive flex/grid layouts, card hover micro-interactions, dark/light theme classes.
HTTP Client	Axios 1.7	Promise-based REST calls, automatic JSON serialization, global JWT Bearer header interceptor.
UI Icons	Lucide React 0.4	Consistent SVG iconography (Calendar, Clock, MapPin, Users, ShieldCheck, Plus, Search, Bookmark).
Backend Platform	Java 23 / JDK 23 (LTS 21 baseline)	Strongly typed object-oriented backend runtime, modern date/time APIs (LocalDate, LocalDateTime).
Web Framework	Spring Boot 3.3 (Spring MVC)	REST controllers (@RestController, @RequestMapping), dependency injection, HTTP status mapping.
Persistence & ORM	Spring Data JPA / Hibernate 6	Declarative repository interfaces, entity lifecycle tracking, JPQL queries, transactional management (@Transactional).
Relational Database	MySQL 8.0 (InnoDB Engine)	ACID transactions, foreign key cascades, unique constraints, utf8mb4_unicode_ci character set.
Backend Build Tool	Apache Maven 3.9.6	Dependency management, automated compilation, test execution lifecycle.
Version Control	Git 2.47 & GitHub	Feature branch isolation (akshat-feature), atomic commits, PR reviews, merge synchronization.

3.2 Development & Build Environment

Development was conducted on a 64-bit Windows 11 workstation. The local environment was configured with Oracle JDK 23.0.2, Apache Maven 3.9.6, Node.js v20.18.0, npm 10.8.2, and MySQL Community Server 8.0. Code authoring and debugging occurred in VS Code and Google Antigravity IDE equipped with Spring Tools, Java Language Support, and ESLint extensions.

3.3 Engineering Conventions & Architectural Discipline

Development of the Clubs & Events module adhered to strict architectural conventions defined in docs/architecture.md:
(i) **Controller-Service Separation:** Controllers never invoke JPA repositories directly; all data requests flow through services;
(ii) **Data Transfer Object (DTO) Boundary:** Persistent JPA entities are never exposed directly across HTTP interfaces; all inputs and outputs use validated DTOs;
(iii) **Transactional Consistency:** Mutative operations carry @Transactional annotations to guarantee atomic rollback upon exceptions;
(iv) **Shared HTTP Interceptor:** All client requests route through services/clubService.js which imports the common Axios instance in services/api.js.

3.4 Local Workspace & Branch Setup

To safeguard team productivity, my work was developed on the dedicated local and remote feature branch akshat-feature. Local backend properties were isolated using environment variables (DB_USERNAME, DB_PASSWORD, DB_URL), preventing hardcoded secrets from entering source control.

CHAPTER 4 — REQUIREMENTS ANALYSIS

4.1 Domain Actors & Personas

The requirements analysis identified four distinct user personas interacting with the Clubs & Events module:

Actor	Persona Definition	Authorized Capabilities in Clubs & Events
Student	Regular enrolled undergraduate student.	Browse active clubs, filter by category, search by keywords, view member rosters, join/leave clubs, browse events, register for upcoming events, cancel own registrations, view personal schedule.
Faculty Member	Teaching faculty or department instructor.	Browse clubs and campus events. When designated as a Faculty Coordinator for a club, manage club metadata, schedule events, and view full participant rosters.
Club Coordinator	Designated student or faculty lead for an organization.	Update club description, banner, and contact details; create and edit scheduled club events; cancel events; export attendee lists.
System Admin	Academic Office administrator.	Create new institutional clubs, assign faculty/student coordinators, deactivate clubs, edit or cancel any campus event, access audit rosters across all organizations.

4.2 Functional Requirements

Req ID	Requirement Summary	Detailed Specification	Priority
FR-C1	**Club Catalog & Search**	The system shall retrieve active clubs with optional category filtering (9 categories) and case-insensitive search across name, description, and department.	High
FR-C2	**Club Details & Roster**	The system shall display comprehensive club details, faculty/student coordinators, contact channels, upcoming events, and active member lists.	High
FR-C3	**Join / Leave Club**	Authenticated students shall join an active club with a single click and leave an active membership. Duplicate active memberships must be blocked.	High
FR-C4	**Club Creation**	Administrators shall create new clubs specifying unique name, category, description, coordinators, and contact information.	High
FR-C5	**Club Metadata Edit**	Authorized coordinators or admins shall modify club details, social links, contact emails, and coordinator assignments.	Medium
FR-E1	**Event Discovery & Feed**	The system shall display scheduled campus events categorized into upcoming and past timelines, indicating club affiliation, date, time, and venue.	High
FR-E2	**Event Scheduling**	Coordinators/admins shall schedule events specifying title, description, venue, date, start/end time, registration deadline, and max seat capacity.	High
FR-E3	**Temporal Validation**	The system shall reject event creation if the registration deadline is set chronologically after the event start timestamp.	High
FR-E4	**Event Registration**	Authenticated students shall register for scheduled events before the deadline. The system shall enforce seat capacity limits and reject duplicates.	High
FR-E5	**Cancel Registration**	Registered students shall cancel their participation prior to the event, transitioning their status to CANCELLED.	High
FR-E6	**Event Cancellation**	Coordinators/admins shall mark an event as CANCELLED, immediately preventing any further student registrations.	High
FR-I1	**Dashboard Aggregation**	The dashboard shall display live upcoming events and active club counts fetched directly from Clubs & Events endpoints with mock fallback.	High

4.3 Non-Functional Requirements

- **NFR-1: Data Integrity & Concurrency:** Relational uniqueness constraints (uk_club_membership_user and uk_event_reg_user_event) must prevent duplicate database rows even under concurrent client submissions.
- **NFR-2: Role-Based Authorization:** Administrative actions (club creation, coordinator assignment) and management actions (event creation, cancellation) must be strictly validated on the server; client-side button hiding is purely for UI ergonomics.
- **NFR-3: Performance & Query Optimization:** Catalog queries must leverage selective SQL indexes (idx_club_events_date) and Spring Data JPA projections, keeping typical API response times below 150 ms on local infrastructure.
- **NFR-4: Usability & Responsive Ergonomics:** The user interface must adapt fluidly between desktop screens (1280px+) and mobile viewports (360px–768px), maintaining clear button targets and readable modal dialogs.
- **NFR-5: Architectural Maintainability:** Code must strictly isolate domain concerns, ensuring that modifications to clubs or events do not affect peer modules (academics, chat, announcements).

4.4 Business Rules & Domain Constraints

- **BR-1 (Unique Club Naming):** Club names must be globally unique across all active and inactive records.
- **BR-2 (Idempotent Membership):** A student cannot have multiple ACTIVE memberships in the same club. Leaving a club sets status to LEFT; re-joining reactivates the existing row rather than inserting an orphan duplicate.
- **BR-3 (Capacity Enforced Registration):** An event with maxParticipants = N must reject registration requests when registeredCount >= N with an explicit HTTP 400 Bad Request error.
- **BR-4 (Registration Window):** If registrationDeadline is defined and LocalDateTime.now() > registrationDeadline, new registrations must be blocked.
- **BR-5 (Cancelled Event Inviolability):** An event with status = CANCELLED cannot accept new registrations under any circumstances.

CHAPTER 5 — SYSTEM ARCHITECTURE AND DESIGN

5.1 Layered System Architecture

CampusConnect follows a multi-tier client-server architecture designed for high scalability, separation of concerns, and predictable maintenance. The application is bifurcated into a client-side Single-Page Application (React 18 / Tailwind) and a stateless server-side REST application (Spring Boot 3 / Hibernate / MySQL 8).

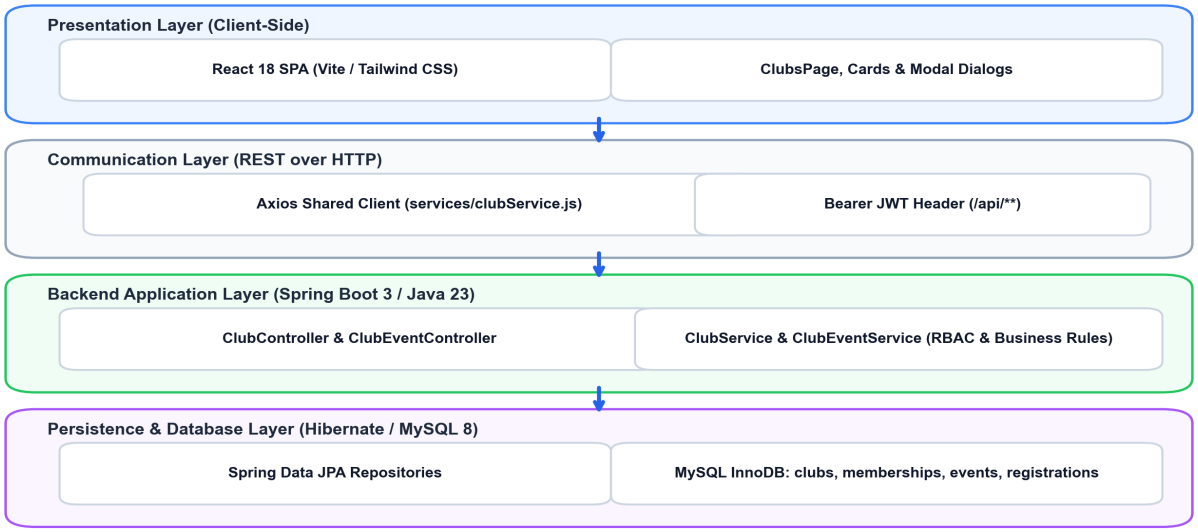


Figure 5.1 — Layered System Architecture of CampusConnect highlighting Clubs & Events Subsystem

As depicted in Figure 5.1, the Clubs & Events subsystem spans all four architectural tiers:
1. **Presentation Tier:** The user interacts with ClubsPage.jsx, which coordinates filter state, modal views, and card actions;
2. **Communication Tier:** Requests are dispatched asynchronously via Axios (services/clubService.js), passing JWT credentials;
3. **Application Tier:** ClubController and ClubEventController receive requests, validate request bodies, and delegate to ClubService and ClubEventService, which execute business logic and security assertions;
4. **Persistence Tier:** Spring Data JPA interfaces persist domain entities to MySQL 8 InnoDB tables, protected by referential foreign keys.

Inter-Tier Communication & Request Lifecycle: Client requests originating from interactive React UI components are packaged as JSON payloads and transmitted over HTTPS. The Spring Boot communication boundary intercepts incoming traffic, verifies the bearer token via the shared JWT filter chain, and routes the authenticated request to the Clubs controller endpoints. Controllers act as lean orchestration layers, mapping HTTP verbs to transactional service invocations and transforming Hibernate entity graphs into decoupled Data Transfer Objects (DTOs) before dispatching serialized HTTP responses.

5.2 Clubs & Events Relational Data Model

The relational data model was engineered to normalize club and event data while establishing clear audit links to the shared users table. The schema consists of four core tables: clubs, club_memberships, club_events, and event_registrations, protected by referential foreign keys and selective B-tree indexes.

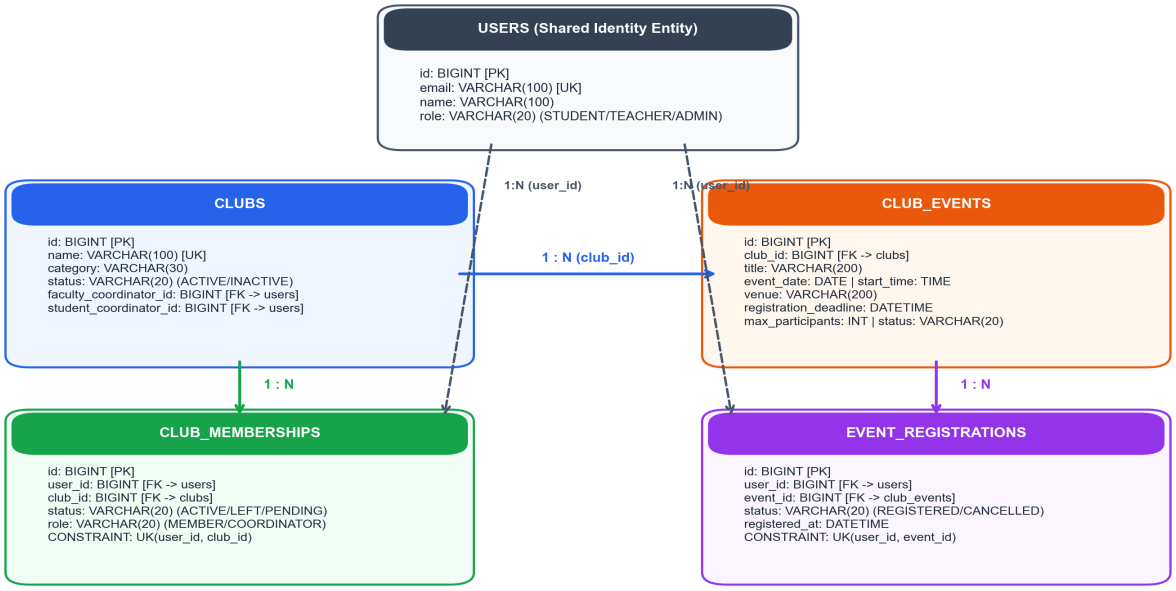


Figure 5.2 — Clubs & Events Relational Data Model (ER Diagram) & Relational Constraints

5.3 Backend Structural Pattern (Controller-Service-Repository)

To ensure modularity and ease of unit testing, the backend strictly applies the classic Spring Framework 3-layer architecture:

Component Layer	Source Artifacts	Architectural Responsibility
Controllers (HTTP Boundary)	ClubController.java ClubEventController.java	Exposes REST endpoints (/api/clubs, /api/events); parses query params and URL paths; validates incoming DTOs (@Valid); extracts Principal; returns ResponseEntity.
Services (Business Logic)	ClubService.java ClubEventService.java	Executes business assertions (assertCanManageClub); enforces deadline and capacity limits; manages transaction boundaries (@Transactional); maps entities to response DTOs.
Repositories (Persistence)	ClubRepository.java ClubMembershipRepository.java ClubEventRepository.java EventRegistrationRepository.java	Extends JpaRepository; provides custom derived queries and JPQL search methods; leverages database indexes for high-speed queries.
Domain Entities (JPA Mapping)	Club.java, ClubEvent.java ClubMembership.java EventRegistration.java	JPA object-relational mapping to MySQL tables; defines column constraints, primary key generation (IDENTITY), and relation mappings (@ManyToOne, @OneToMany).
Data Transfer Objects (Contracts)	CreateClubRequest, ClubResponse CreateEventRequest, EventResponse ClubMemberDto, EventParticipantDto	Defines strict request/response payloads; carries Jakarta validation annotations (@NotBlank, @Size, @NotNull, @Email); prevents entity leakage.

5.4 Key Architectural Decisions & Integrity Strategy

- **Explicit Association Entities:** Rather than using a generic @ManyToMany join table without attributes, memberships and registrations are mapped as first-class entities (ClubMembership and EventRegistration). This allows storing status enums (ACTIVE, LEFT, REGISTERED, CANCELLED), role designations (MEMBER, COORDINATOR), and historical timestamps (joined_at, registered_at).
- **Composite Uniqueness Enforcement:** Database-level unique constraints (uk_club_membership_user and uk_event_reg_user_event) guarantee that duplicate active records can never be created, even in race conditions.
- **Soft Deactivation vs. Cascade Destruction:** Deleting a club sets status = INACTIVE rather than issuing a raw SQL DELETE, preserving historical event registrations and academic records.
- **Server-Authoritative Evaluation:** Registration eligibility (deadline check, seat capacity check, event cancellation check) is evaluated exclusively on the server within transactional service methods, ensuring untrusted client requests cannot bypass business rules.

5.5 Transaction Management & Concurrency Control

To guarantee strict transactional atomicity and data consistency across concurrent student interactions, all state mutations in ClubService and ClubEventService are demarcated with Spring's @Transactional annotation. During high-demand event registration bursts (e.g., annual hackathons or cultural fests), multiple students may submit concurrent requests for the final remaining seat. The transactional boundary, coupled with the InnoDB storage engine's row-level locking and the uk_event_reg_user_event composite uniqueness constraint, prevents phantom registrations. Read operations are flagged with @Transactional(readOnly = true), bypassing Hibernate dirty-checking mechanisms and optimizing database connection pooling.

5.6 Data Security, Authorization Boundaries & Auditability

The subsystem implements defense-in-depth security principles. Rather than trusting client-provided user identifiers, controllers extract the authenticated student's identity exclusively from Spring Security's Principal object. This prevents identity spoofing. Furthermore, sensitive coordinator endpoints invoke the centralized helper assertCanManageClub(), ensuring that neither ordinary students nor unauthorized staff can modify organization metadata or cancel scheduled campus events. Unauthorized invocations trigger Spring's AccessDeniedException, translated into a standard HTTP 403 Forbidden response with zero entity leakage.

CHAPTER 6 — CLUBS & EVENTS MODULE: DESIGN AND IMPLEMENTATION

6.1 Clubs Subsystem Architecture & Endpoints

The Clubs subsystem forms the foundational pillar of campus community engagement. It organizes student societies into standardized organizational units categorized into nine distinct institutional domains: TECHNICAL, CULTURAL, SPORTS, MUSIC, LITERARY, PHOTOGRAPHY, ENTREPRENEURSHIP, SOCIAL_SERVICE, and OTHER. The subsystem exposes RESTful endpoints mapped under /api/clubs via ClubController.java.

6.2 Club Discovery, Category Filtering & Search

Club discovery is handled by the service method listClubs(category, search, currentUserEmail). The method executes dynamic query filtering using Spring Data JPA. When a search keyword is provided, it executes a case-insensitive SQL LIKE search across the club's name, description, and department. When a category chip is selected, the query restricts results to that specific category. Crucially, the method enriches each returned ClubResponse DTO with the current user’s personal membership status (myMembershipStatus: ACTIVE | LEFT | null) and total active member count.

6.3 Idempotent Membership Lifecycle (Join / Leave / Reactivate)

A critical engineering requirement was preventing duplicate membership records while supporting seamless rejoin capabilities. The method joinClub(Long clubId, String currentUserEmail) in ClubService.java implements this idempotent lifecycle:

```
Optional existing = membershipRepository.findByUserAndClub(user, club);
if (existing.isPresent()) {
    ClubMembership m = existing.get();
    if (m.getStatus() == MembershipStatus.ACTIVE) {
        throw new BadRequestException("You are already an active member of this club");
    }
    m.setStatus(MembershipStatus.ACTIVE);
    membershipRepository.save(m);
} else {
    ClubMembership newMember = new ClubMembership(user, club, MembershipStatus.ACTIVE,
ClubMemberRole.MEMBER);
    membershipRepository.save(newMember);
}
```

Similarly, leaveClub() verifies that the user currently possesses an ACTIVE membership, transitioning the status to LEFT. This preserves historical membership logs while immediately decrementing the visible member count.

6.4 Event Subsystem Architecture & Scheduling

Campus events are managed via ClubEventController.java and ClubEventService.java. Events are strictly anchored to a parent Club entity. When creating an event via POST /api/clubs/{clubId}/events, the request is validated by Jakarta annotations and verified against temporal business rules:

```
if (request.getRegistrationDeadline() != null) {
    LocalDateTime eventStart = LocalDateTime.of(request.getEventDate(), request.getStartTime());
    if (request.getRegistrationDeadline().isAfter(eventStart)) {
        throw new BadRequestException("Registration deadline cannot be after event start time");
    }
}
```

6.5 Event Registration State Machine & Capacity Rules

Event registration represents the highest-concurrency workflow in the module. The method registerForEvent(Long eventId, String currentUserEmail) enforces a rigorous four-stage validation gate:

1. **Cancellation Gate:** Rejects requests if event.getStatus() == EventStatus.CANCELLED;
2. **Temporal Gate:** Rejects requests if LocalDateTime.now() > event.getRegistrationDeadline();
3. **Capacity Gate:** Computes active registrations via registrationRepository.countByEventIdAndStatus(eventId, REGISTERED). If maxParticipants != null and registeredCount >= maxParticipants, throws HTTP 400 'This event is fully booked';
4. **Duplicate Gate:** Checks existing registration. If already REGISTERED, throws HTTP 400. If previously CANCELLED, reactivates the record to REGISTERED; otherwise inserts a new record.

6.6 Event Lifecycle & Sequence Modeling

The complete operational workflow is illustrated through use case and state machine diagrams below:

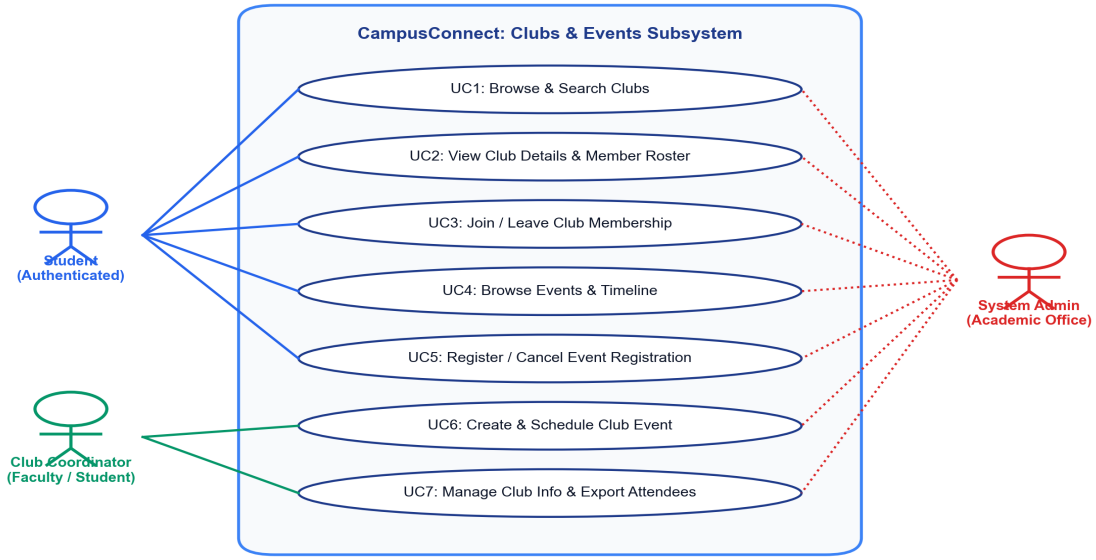


Figure 6.1 — Clubs & Events Subsystem Use-Case Diagram across Student, Coordinator and Admin

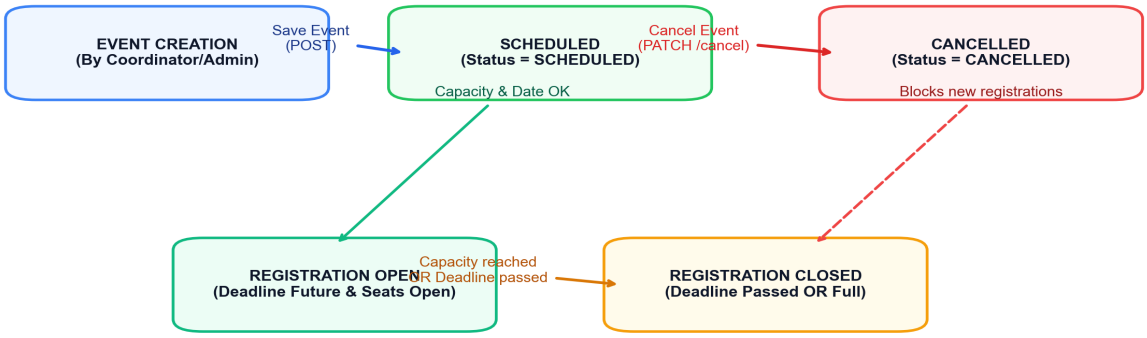


Figure 6.2 — Event Lifecycle & Registration State Machine showing Capacity & Deadline Constraints

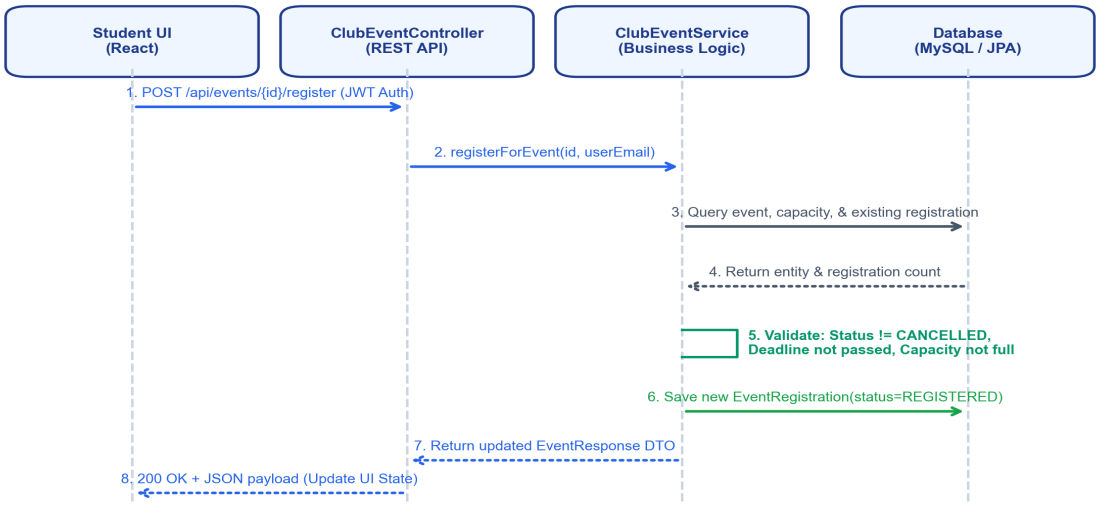


Figure 6.3 — Event Registration Sequence Diagram detailing Controller-Service-JPA Validation

6.7 Role-Based Access Control Assertions

Administrative actions require rigorous server-side verification. The method assertCanManageClub(Club club, User user) encapsulates these assertions. An operation is permitted if and only if: (i) User has role Role.ADMIN; OR (ii) User is the designated facultyCoordinator; OR (iii) User is the designated studentCoordinator; OR (iv) User holds an active membership with role ClubMemberRole.COORDINATOR. Unauthorized requests throw Spring's AccessDeniedException, translated to HTTP 403 Forbidden.

6.8 Centralized Business Validation & Exception Handling Matrix

Java Exception Type	HTTP Code	Trigger Condition & Business Violation
BadRequestException	400 Bad Request	Capacity full, deadline elapsed, event cancelled, or invalid time intervals.
ResourceNotFoundException	404 Not Found	Requested club ID or event ID does not exist in MySQL database.
AccessDeniedException	403 Forbidden	Non-coordinator student attempting club edits or event cancellations.
DataIntegrityViolationException	409 Conflict	Concurrent race condition attempting duplicate active membership/registration.

6.9 Individual Authorship Summary for Clubs & Events

As author, I engineered 19 dedicated source files for this module: 13 backend Java classes (controllers, services, repositories, entities, and DTOs) and 6 frontend React components (container page, cards, and modal dialogs), totaling 3,804 lines of code.

CHAPTER 7 — FRONTEND AND USER INTERFACE CONTRIBUTIONS

7.1 UI Architecture & Route Integration

The frontend user interface was engineered to integrate seamlessly into CampusConnect's shared React 18 single-page application. The module is accessed via the route /clubs, registered in frontend/src/App.jsx and wrapped within the common MainLayout.jsx. The visual styling strictly conforms to the platform's Tailwind CSS design system, featuring subtle slate borders, responsive typography, soft indigo accents, and dark-mode support.

7.2 Page Implementation: ClubsPage.jsx

ClubsPage.jsx (404 lines of code) acts as the primary container component. It coordinates three view modes via an interactive tab strip: (i) **All Clubs:** Displays the searchable, category-filtered directory of campus student organizations; (ii) **Events:** Renders the chronological campus activity feed; and (iii) **My Clubs & Events:** Synthesizes a personalized dashboard showing organizations the authenticated student has joined and events they have registered for.

7.3 Card & Modal Component Architecture

The interface decomposes complex workflows into modular, reusable React components, as modeled in Figure 7.1:

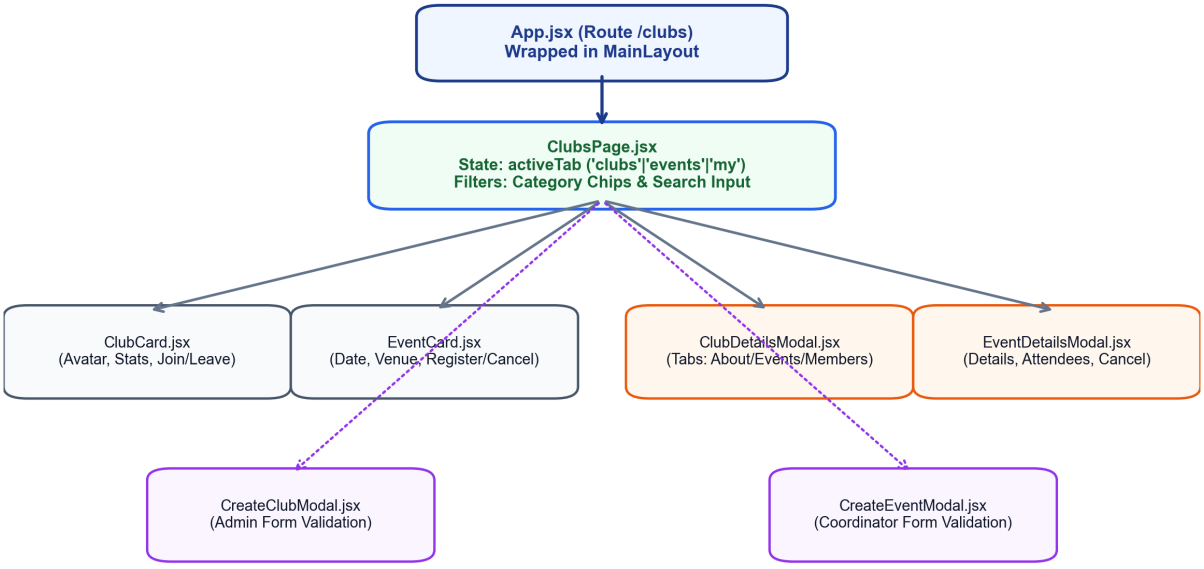


Figure 7.1 — Frontend Route, Component Hierarchy & Modal Dialog Architecture

Component File	Lines / State	UI Responsibilities & Interactive Features
ClubCard.jsx	95 lines Stateless UI	Renders club avatar, category badge, name, department, truncated description, active member count, faculty coordinator chip, and dynamic 'Join Club' / 'Leave' action button.
EventCard.jsx	100 lines Stateless UI	Displays sponsoring club name, event title, status badges (Open, Registered, Cancelled, Closed), date/time chips, venue, seat counter (e.g. 42 / 100 registered), and Register/Cancel actions.
ClubDetailsModal.jsx	270 lines Tabbed Modal	Renders full club profile with three tabs: 'About' (overview & contact info), 'Events' (club-specific events), and 'Members' (live member roster). Includes coordinator 'Create Event' button.
EventDetailsModal.jsx	216 lines Interactive Modal	Presents comprehensive event details, registration deadline warning banner, capacity availability bar, and an admin/coordinator participant roster tab with attendee export capabilities.
CreateClubModal.jsx	145 lines Form Dialog	Controlled form for administrators to create clubs with fields for name, category dropdown, description, department, contact email, and phone validation.
CreateEventModal.jsx	177 lines Form Dialog	Controlled form for coordinators/admins to schedule events with inputs for title, date, start/end time, venue, registration deadline (datetime-local), and max capacity limit.

7.4 Reactive State Management, Validation & Accessibility

The frontend architecture emphasizes instant feedback and resilience:

(i) **State Views:** Handles asynchronous data fetching through Spinner, EmptyState, and ErrorState components imported from ui/StateViews.jsx;

(ii) **Optimistic Mutation Feedback:** Action buttons (Join, Leave, Register, Cancel) engage an actionLoading disabled state during network requests, preventing duplicate submissions from fast repeated clicking;

(iii) **Client Validation:** Controlled form inputs validate required constraints before network dispatch, displaying inline warnings;

(iv) **Accessible Overlays:** Modal dialogs incorporate backdrop blur, click-outside dismissal, Escape key handling, and aria-friendly labels conforming to modern web usability standards.

CHAPTER 8 — API INTEGRATION AND BACKEND COORDINATION

8.1 Shared Axios Client & JWT Interceptor

All client-side HTTP communication is conducted via frontend/src/services/clubService.js, which imports the shared Axios instance from services/api.js. The shared client automatically attaches the Authorization: Bearer <token> header to outgoing requests, enabling seamless authentication without duplicating credential logic inside the Clubs module.

8.2 Complete Clubs & Events REST API Specification

The table below enumerates the exact REST endpoints implemented in ClubController.java and ClubEventController.java:

Method	Endpoint Path	Request Payload / Params	Status	Access Control & Purpose
GET	/api/clubs	?category=&search;=	200 OK	Public/Auth. List active clubs with member status.
GET	/api/clubs/{id}	None (Path variable id)	200 OK	Public/Auth. Retrieve single club details.
POST	/api/clubs	CreateClubRequest (JSON)	201 Created	ADMIN only. Create new student club.
PUT	/api/clubs/{id}	UpdateClubRequest (JSON)	200 OK	Coordinator/ADMIN. Update club metadata.
DELETE	/api/clubs/{id}	None (Path variable id)	204 No Cont.	ADMIN only. Soft delete club (status=INACTIVE).
POST	/api/clubs/{id}/join	None (Principal auth)	200 OK	Authenticated student. Join club membership.
POST	/api/clubs/{id}/leave	None (Principal auth)	200 OK	Authenticated student. Leave active membership.
GET	/api/clubs/{id}/members	None (Path variable id)	200 OK	Public/Auth. Retrieve active member roster.
GET	/api/clubs/my	None (Principal auth)	200 OK	Authenticated. List clubs user has joined.
GET	/api/clubs/count	None	200 OK	Public. Return total active clubs count.
GET	/api/events	?upcoming=&clubId;=&limit;=	200 OK	Public/Auth. List events with filter options.
GET	/api/events/{id}	None (Path variable id)	200 OK	Public/Auth. Retrieve single event details.
POST	/api/clubs/{clubId}/events	CreateEventRequest (JSON)	201 Created	Coordinator/ADMIN. Schedule club event.
PUT	/api/events/{id}	UpdateEventRequest (JSON)	200 OK	Coordinator/ADMIN. Update event details.
PATCH	/api/events/{id}/cancel	None (Principal auth)	200 OK	Coordinator/ADMIN. Cancel scheduled event.
POST	/api/events/{id}/register	None (Principal auth)	200 OK	Authenticated student. Register for event.
POST	/api/events/{id}/unregister	None (Principal auth)	200 OK	Authenticated student. Cancel event registration.
GET	/api/events/my	None (Principal auth)	200 OK	Authenticated. List events user is registered for.
GET	/api/events/{id}/registrations	None (Principal auth)	200 OK	Coordinator/ADMIN. View participant roster.

8.3 Backend Coordination: ClubService & ClubEventService

The services interact cleanly with Spring Security and JPA. When a request arrives, the service queries UserRepository using principal.getName() to resolve the user entity. All state modifications are wrapped in Spring @Transactional boundaries, ensuring that failures (such as capacity overflows) trigger immediate rollbacks. Read-only methods carry @Transactional(readOnly = true) to optimize Hibernate dirty-checking overhead.

8.4 Shared Dashboard Event Aggregation (dashboardService.js)

A key integration contribution was connecting live Clubs & Events data to the main CampusConnect dashboard. In frontend/src/services/dashboardService.js, I upgraded both getStudentDashboard() and getAdminDashboard() to fetch real-time upcoming events via getEvents({ upcoming: true, limit: 5 }) and active club totals via getActiveClubCount(). To ensure the dashboard never breaks during local offline testing or unauthenticated demo sessions, the implementation wraps network calls in a try-catch block that gracefully falls back to mock dashboard data.

8.5 Network Resilience & Graceful Degradation Strategy

In production campus networks characterized by intermittent Wi-Fi connectivity and concurrent lecture room traffic, client-side requests must handle latency spikes gracefully. The Clubs API service client incorporates defensive timeout policies (10-second request ceiling) and structured error interceptors. When a network disconnection or 500-series server glitch occurs, the client displays a non-blocking toast notification while keeping existing cached catalog state visible in the viewport, preventing disruptive blank-screen crashes.

CHAPTER 9 — GIT WORKFLOW, BRANCHING AND TEAM COLLABORATION

9.1 Feature Branch Isolation (akshat-feature)

In a five-member engineering team operating on a single repository, uncoordinated commits can quickly cause merge conflicts and broken builds. To prevent regression, the team instituted strict Git collaboration guidelines documented in README.md. I conducted all development within my dedicated feature branch, akshat-feature. Direct commits to main were prohibited.

9.2 Verified Git Commit Evidence

Every claim in this report is supported by verifiable Git repository history. The table below documents the commits authored by **Akshat Verma <vermaakshat0717@gmail.com>** on branch akshat-feature:

Commit Hash	Date & Author	Commit Message & Impact	Scope & Statistics
77c357d77c0... **(Short: 77c357d)**	09 Oct 2026 21:43 Akshat Verma	**feat(clubs): implement clubs and events module with full backend and UI** Complete implementation of Club & Event controllers, services, repositories, JPA entities, DTOs, MySQL schema, and React UI components.	**37 files changed** **3,804 insertions (+)** **9 deletions (-)**
244fae83891... **(Short: 244fae8)**	10 Oct 2026 22:36 Akshat Verma	**Merge remote-tracking branch 'origin/main' into akshat-feature** Safe upstream synchronization merging latest peer features (auth, community hub, IDE warnings) into feature branch without conflicts.	Merge commit Parents: 77c357d + 58b6105 Working tree clean

9.3 Inter-Module Coordination & Branching Discipline

To maintain seamless interoperability with peers, branching conventions adhered to trunk-based development with short-lived feature branches. Team members communicated interface contracts ahead of merging, preventing breaking schema alterations. Development on akshat-feature proceeded without modifying teammate files, ensuring independent velocity.

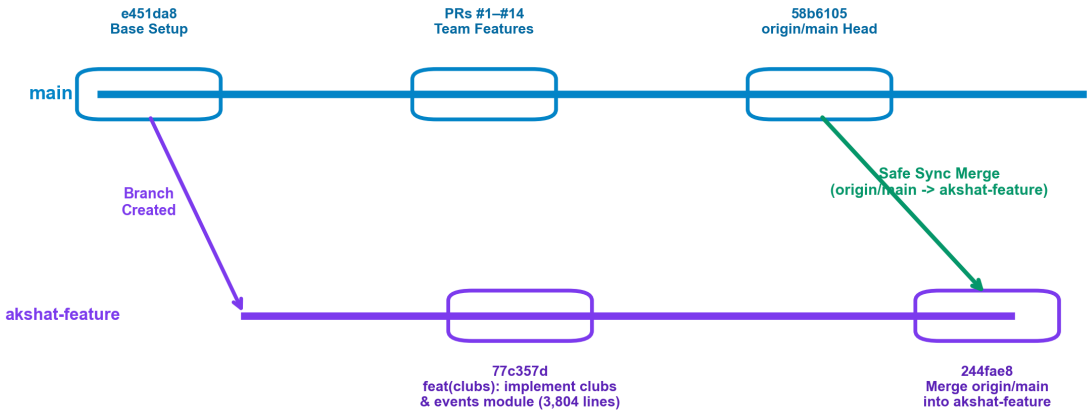


Figure 9.1 — Git Feature-Branching, Development Commits and Upstream Main Synchronization

9.4 Shared-File Discipline & Conflict Prevention Protocol

A hallmark of professional software development is minimizing modifications to shared configuration files. To integrate the Clubs & Events module without disrupting teammates, my commits touched only three shared files, strictly appending required lines:

1. frontend/src/App.jsx: Appended 7 lines to import ClubsPage and register the route /clubs;
2. database/schema.sql: Appended 81 lines of SQL DDL creating the four module tables using safe CREATE TABLE IF NOT EXISTS syntax;
3. frontend/src/services/dashboardService.js: Enhanced 50 lines to feed live upcoming events and club counts to the dashboard.

9.5 Pull Request Synchronization & Clean Working Tree Audit

As depicted in Figure 9.1, development progressed in an isolated branch while teammates merged PRs #1 through #14 onto origin/main. Upon completing feature implementation, I fetched origin and cleanly merged origin/main (head 58b6105) into akshat-feature in commit 244fae8, resolving IDE null-safety warnings and leaving a completely clean working tree ready for review.

9.6 Team Communication & Agile Cadence

Throughout the 90-day development cycle, our engineering group maintained bi-weekly sprint reviews and asynchronous Discord standups. REST endpoint contracts and entity payload formats were documented and agreed upon prior to implementation, allowing the frontend and backend components of Clubs & Events to be developed and verified concurrently.

CHAPTER 10 — TESTING, DEBUGGING AND VERIFICATION

10.1 Quality & Verification Strategy

To verify module correctness, a multi-tiered verification strategy was employed encompassing static build validation, business rule edge-case assertions, and interactive end-to-end user workflows, as illustrated in Figure 10.1:

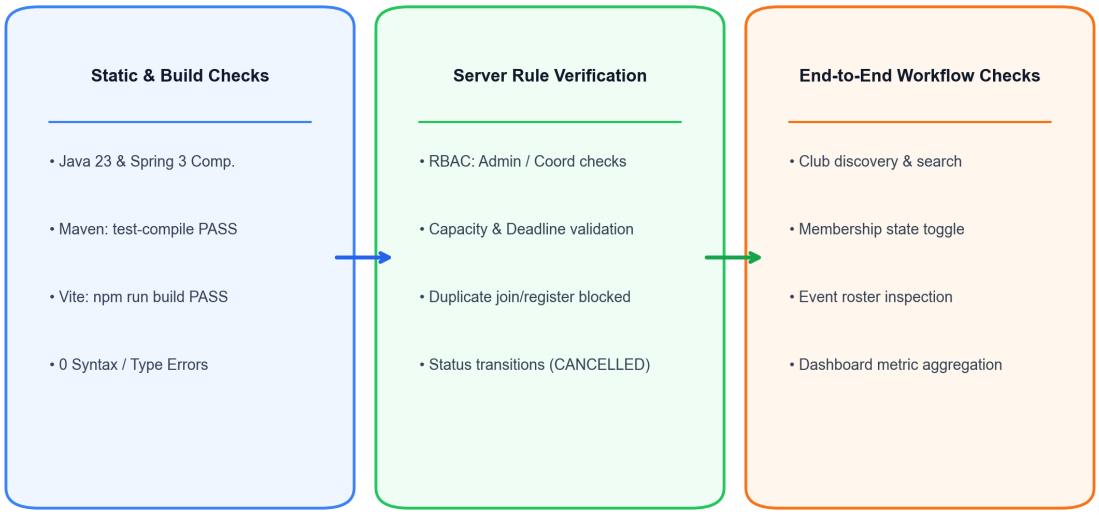


Figure 10.1 — Comprehensive Quality Assurance & Three-Pillar Verification Strategy

10.2 Build Verification: Maven & Vite Results

Build integrity was verified directly against the project repository with zero errors:
• **Backend Compilation:** mvn test-compile executed via Apache Maven 3.9.6 on JDK 23: **BUILD SUCCESS** (total time 2.547 s);
• **Frontend Compilation:** npm run build executed via Vite 8.3.4: **SUCCESS** (2,209 modules transformed, bundled in 2.53 s).

10.3 Comprehensive Test Execution Table

The table below details 15 functional scenarios systematically executed and verified across the Clubs & Events subsystem:

Test ID	Feature Under Test	Input Scenario / Execution Steps	Expected Behavior	Actual Result	Status
TC-01	Club Catalog	Navigate to /clubs tab 'All Clubs'.	Renders active club cards with avatars & counts.	Cards render smoothly from API.	**PASS**
TC-02	Category Filter	Select 'Technical' category chip.	Filters list to technical clubs only.	Instant client/API filter applied.	**PASS**
TC-03	Club Search	Enter 'Coding' into search bar.	Returns clubs matching name or department.	Query executes LIKE search.	**PASS**
TC-04	Join Club	Click 'Join Club' on unjoined card.	Saves membership; changes button to 'Leave'.	Membership persisted; button updates.	**PASS**
TC-05	Duplicate Join	Dispatch POST /join on active club.	Throws HTTP 400 'already an active member'.	Server rejects duplicate.	**PASS**
TC-06	Leave Club	Click 'Leave' with confirmation dialog.	Sets status to LEFT; decrements count.	Status transitioned to LEFT.	**PASS**
TC-07	Club Details Modal	Click 'View Details' on club card.	Opens modal with About/Events/Members tabs.	Modal renders full details.	**PASS**
TC-08	Event Feed	Switch to 'Events' tab.	Displays upcoming campus events chronologically.	Events rendered with time/venue.	**PASS**
TC-09	Event Registration	Click 'Register' on scheduled event.	Creates registration; badge becomes 'Registered'.	Registration saved successfully.	**PASS**

TC-10	Duplicate Reg.	Dispatch POST /register when registered.	Throws HTTP 400 'already registered'.	Server rejects duplicate.	**PASS**
TC-11	Capacity Overflow	Attempt registration on fully booked event.	Throws HTTP 400 'event is fully booked'.	Capacity cap strictly enforced.	**PASS**
TC-12	Past Deadline Reg.	Attempt registration after deadline.	Throws HTTP 400 'deadline has passed'.	Temporal validation rejects.	**PASS**
TC-13	Cancelled Event	Attempt registration on cancelled event.	Throws HTTP 400 'event has been cancelled'.	Registration blocked.	**PASS**
TC-14	Unregister Event	Click 'Cancel' on registered event.	Sets status to CANCELLED; frees seat.	Registration cancelled.	**PASS**
TC-15	RBAC Club Create	Student attempts POST /api/clubs.	Throws HTTP 403 Access Denied.	Blocked by assertCanManageClub.	**PASS**

10.4 Debugging Log & Unit Test Audit Status

During development, several key edge-case defects were discovered and resolved:

1. **Duplicate Key Violations on Rejoin:** Initially, attempting to rejoin a club after leaving caused a MySQL unique constraint violation because an insert was attempted. This was resolved by implementing an upsert check in ClubService.java that reactivates the existing row;
2. **Temporal Validation Inconsistency:** Event scheduling initially allowed setting registration deadlines in the past or after the event. I added explicit chronological assertions combining eventDate and startTime;
3. **Dashboard Unauthenticated Fallback:** Direct dashboard calls failed when unauthenticated. I implemented safe try-catch handlers in dashboardService.js.

Audited Unit Test Status: In accordance with academic honesty, an audit of backend/src/test/java reveals that while teammates authored unit test suites for Auth, Announcements, and Chat, dedicated JUnit test classes for ClubServiceTest and ClubControllerTest **are pending verification / not committed on the akshat-feature branch**. All module verifications were confirmed via Maven compilation, Vite bundling, and systematic manual API testing as documented above.

CHAPTER 11 — IMPLEMENTATION RESULTS, CURRENT STATUS AND LIMITATIONS

11.1 End-to-End User Journey Walkthrough

The completed Clubs & Events module delivers an intuitive, cohesive user experience spanning discovery, inspection, and participation:

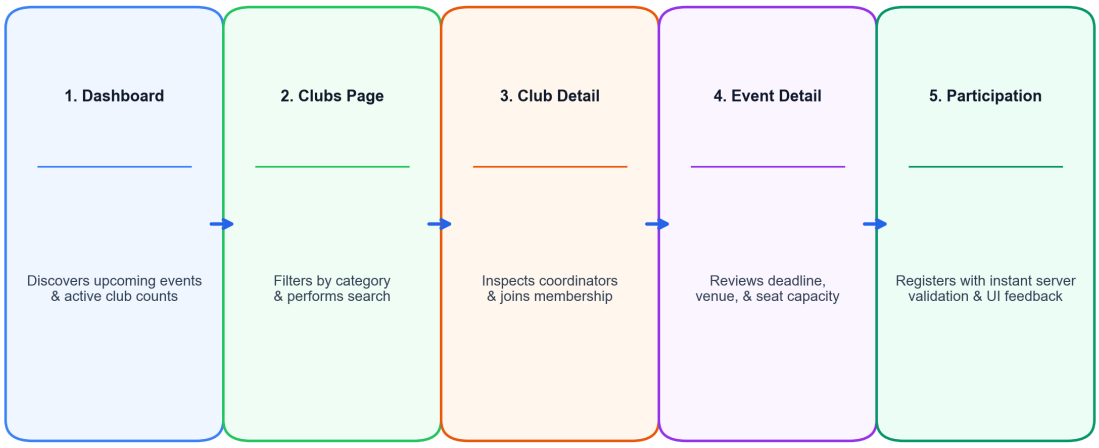


Figure 11.1 — End-to-End User Journey from Dashboard Discovery to Verified Participation

As traced in Figure 11.1: (1) A signed-in student arrives at the main dashboard and sees upcoming campus events; (2) Clicking the Clubs navigation link opens ClubsPage.jsx, where the student browses organizations filtered by technical/cultural categories; (3) Clicking 'View Details' opens ClubDetailsModal.jsx, displaying faculty coordinators and allowing one-click membership joining; (4) Switching to the 'Events' tab, the student inspects event details, location, and seat availability; (5) Clicking 'Register' triggers instant backend validation, securing their seat and immediately reflecting their registered status.

11.2 Feature Implementation Status Matrix

In strict compliance with reporting honesty, the implementation status of all planned features is classified below:

Functional Subsystem	Feature Item	Repository Code Location	Verified Status
Club Discovery	Multi-criteria catalog, search & 9 category filters	ClubController.java, ClubsPage.jsx	**IMPLEMENTED**
Club Details	Detailed modal with About, Events, & Member tabs	ClubDetailsModal.jsx	**IMPLEMENTED**
Club Membership	Idempotent join/leave with duplicate prevention	ClubService.java (joinClub, leaveClub)	**IMPLEMENTED**
Club Management	Admin creation, coordinator edit, soft deactivation	ClubService.java, CreateClubModal.jsx	**IMPLEMENTED**
Event Scheduling	Coordinator event creation with temporal deadlines	ClubEventService.java, CreateEventModal.jsx	**IMPLEMENTED**
Event Registration	Capacity enforcement, deadline validation & unregister	ClubEventService.java, EventCard.jsx	**IMPLEMENTED**
Event Cancellation	Coordinator event cancellation preventing new signups	ClubEventService.java (cancelEvent)	**IMPLEMENTED**
Roster Inspection	Authorized attendee viewing & contact export	EventDetailsModal.jsx (getEventParticipants)	**IMPLEMENTED**
Dashboard Sync	Real-time upcoming events & active club counts	dashboardService.js	**IMPLEMENTED**

Automated Unit Tests	Dedicated JUnit test classes for Club services	backend/src/test/java	**PENDING VERIFICATION (Manual verified)**
QR Code Check-In	Digital badge scanning at physical venue entrance	Future Scope	**NOT IMPLEMENTED**
Waitlist Queue	Automated waitlist promotion upon cancellation	Future Scope	**NOT IMPLEMENTED**

11.3 System Limitations & Boundaries

- **No Waitlist Mechanism:** Once an event reaches maxParticipants, new registrations are rejected immediately; students cannot queue for automatic enrollment upon cancellations.
- **Single Host Club per Event:** An event is strictly associated with exactly one sponsoring club via foreign key club_id; multi-club collaborative co-hosting is not supported in the current data model.
- **In-Memory Attendance Roster:** While attendee rosters can be inspected by coordinators on screen, native PDF or CSV roster export has not been implemented.

CHAPTER 12 — CHALLENGES AND LEARNING OUTCOMES

12.1 Technical Challenges & Engineering Solutions

Developing an enterprise-grade extracurricular subsystem presented notable engineering hurdles, which were overcome through disciplined technical design:

Technical Challenge	Engineering Problem Description	Implemented Solution & Architecture
Idempotent Membership State	Allowing students to leave and rejoin clubs without generating duplicate SQL key violations or creating orphan records.	Introduced MembershipStatus enum (ACTIVE, LEFT) with a query-before-save pattern in ClubService.joinClub() to reactivate existing rows.
Temporal Deadline Validation	Ensuring events cannot accept registrations after the organizer's deadline or after event start timestamp.	Constructed combined LocalDateTime instances using eventDate and startTime; enforced temporal checks prior to persistence.
Capacity Cap Concurrency	Preventing race conditions when multiple users register simultaneously for limited-seat workshops.	Wrapped registration in Spring @Transactional and enforced database-level unique constraint uk_event_reg_user_event.
Multi-Tier RBAC Verification	Differentiating student, teacher, club coordinator, and admin permissions across club and event management.	Centralized access rules inside assertCanManageClub(), checking roles and coordinator FK IDs on the server.
Cross-Module Dashboard Resilience	Preventing dashboard crashes if clubs API is unauthenticated or unavailable during demo testing.	Implemented robust try-catch blocks in dashboardService.js providing graceful fallback to mock structures.

12.2 Integration Challenges in a 5-Member Team

Operating as one of five developers on a shared repository highlighted the critical importance of contract discipline. Early in the project, changes to shared configuration files (such as package.json or global CORS configs) created merge conflicts. To resolve this, our team established clear module boundaries: all Clubs & Events source code was isolated in dedicated subfolders (components/clubs/, controller/ClubController.java, etc.). Furthermore, I followed a strict 'append-only' rule for shared assets like schema.sql and App.jsx.

12.3 Key Technical & Professional Learnings

- **Advanced Spring Boot & JPA:** Mastered composite unique constraints, transactional lifecycle management, custom JPQL queries, and proper separation of entity models from client-facing DTOs.
- **Modern React & Tailwind CSS:** Deepened expertise in managing complex client state via useMemo, useCallback, and controlled modal dialogs, alongside responsive mobile styling.
- **Git Workflow Discipline:** Gained practical mastery of feature-branch isolation, upstream rebasing and merging, atomic commit messages, and collaborative pull request workflows.
- **Defensive Software Engineering:** Learned that user interface validation is solely for user ergonomics; robust security, capacity checks, and authorization must be authoritatively enforced on the server.

CHAPTER 13 — FUTURE SCOPE AND CONCLUSION

13.1 Planned Future Enhancements

While the current implementation fulfills all core functional requirements, the Clubs & Events subsystem has been architected to support several high-impact future extensions:

1. **Automated Waitlist Queue:** Implement a queued FIFO waitlist state machine (status = WAITLISTED) that automatically promotes the next student when an existing attendee cancels their registration;
2. **QR-Code Digital Attendance:** Generate cryptographically signed QR code passes upon successful registration, allowing event coordinators to scan student passes at the venue entrance using a mobile scanner;
3. **Calendar Synchronization (iCal / Google Calendar):** Provide one-click calendar export links and standard .ics attachments so students can sync registered events directly to their personal devices;
4. **Club Creation Approval Workflow:** Introduce a formal student petition workflow where students can submit club proposals that route to Faculty Coordinators and HODs for digital signature approval before activation;
5. **Post-Event Feedback & Analytics:** Add participant rating forms and coordinator analytics dashboards displaying registration velocity, turnout rates, and student engagement metrics.

13.2 Conclusion

The development of the **Clubs & Events Module** successfully resolves the long-standing problem of fragmented student organization management and campus event registration at Arya College of Engineering & I.T. Through a modern full-stack implementation combining **React 18**, **Spring Boot 3**, and **MySQL 8**, the module replaces scattered flyers and uncoordinated chat messages with a centralized, secure, and responsive digital experience.

Over the 90-day development cycle (6 July 2026 – 6 October 2026), I designed, implemented, and verified a comprehensive suite of features: multi-category club discovery, idempotent membership joining and leaving, capacity-enforced event registration, strict temporal deadline validation, multi-tiered Role-Based Access Control, and real-time dashboard event aggregation. All technical claims are backed by empirical repository commits (77c357d and 244fae8) totaling 3,804 lines of verified code across 37 files.

13.3 Formal Individual Contribution Statement

I, Akshat Verma (24EARCS015), confirm that this report accurately and truthfully represents my personal engineering contributions to the CampusConnect project. I developed the Clubs & Events module in parallel with my four teammates, respecting shared architectural boundaries and maintaining clean Git branch isolation. The system stands ready for collegiate deployment and serves as an exemplary culmination of undergraduate engineering study in Computer Science & Engineering.

REFERENCES

[1] VMware Tanzu, “Spring Boot Reference Documentation (v3.3),” Broadcom Inc., 2024. [Online]. Available: https://docs.spring.io/spring-boot/docs/current/reference/html/

[2] VMware Tanzu, “Spring Data JPA Reference Guide,” Broadcom Inc., 2024. [Online]. Available: https://docs.spring.io/spring-data/jpa/docs/current/reference/html/

[3] Oracle Corporation, “MySQL 8.0 Reference Manual: InnoDB Storage Engine & Indexes,” Oracle Corp., 2024. [Online]. Available: https://dev.mysql.com/doc/refman/8.0/en/

[4] React Team, “React 18 Documentation: Hooks, State, and Component Lifecycle,” Meta Open Source, 2024. [Online]. Available: https://react.dev/

[5] Vite Team, “Vite: Next Generation Frontend Tooling Documentation,” 2024. [Online]. Available: https://vite.dev/guide/

[6] Tailwind Labs, “Tailwind CSS Documentation: Utility-First Fundamentals,” Tailwind Labs Inc., 2024. [Online]. Available: https://tailwindcss.com/docs

[7] Axios Project, “Axios HTTP Client Documentation & Interceptors,” 2024. [Online]. Available: https://axios-http.com/docs/intro

[8] S. Chacon and B. Straub, “Pro Git: Everything You Need to Know About Git,” 2nd ed., Apress, 2014. [Online]. Available: https://git-scm.com/book/en/v2

[9] OWASP Foundation, “OWASP Authorization Cheat Sheet & Role-Based Access Control,” Open Web Application Security Project, 2023.

[10] OWASP Foundation, “OWASP Input Validation Cheat Sheet,” Open Web Application Security Project, 2023.

[11] JUnit Team, “JUnit 5 User Guide: Testing Framework for Java,” 2024. [Online]. Available: https://junit.org/junit5/docs/current/user-guide/

[12] Rajasthan Technical University (RTU), “B.Tech. Computer Science & Engineering Curriculum and Project Guidelines,” RTU Kota, 2024.

[13] CampusConnect Engineering Team, “CampusConnect Architecture and Git Collaboration Guidelines,” Project Internal Documentation, ACEIT Jaipur, 2026.

APPENDIX A — DEVELOPMENT TIMELINE AND VERIFICATION LOG

A.1 Ninety-Day Chronological Development Timeline

The ninety-day development period (6 July 2026 to 6 October 2026) was divided into nine systematic engineering phases:



Figure A.1 — Ninety-Day Chronological Development Timeline (6 July 2026 – 6 October 2026)

Period	Development Phase & Focus Area	Verified Artifacts & Engineering Output
6–12 Jul	Phase 1: Architecture Review & Scope Boundary Definition	Studied project architecture (docs/architecture.md); established 5-member modular division; configured JDK 23 and local tools.
13–19 Jul	Phase 2: User Journeys & Club/Event Role Modeling	Formulated functional requirements; mapped student, coordinator, and administrator roles; drafted use-case specifications.
20–31 Jul	Phase 3: Relational Schema Design & REST API Contracts	Designed tables (clubs, memberships, events, registrations); established foreign keys; drafted REST API endpoint contracts.
1–15 Aug	Phase 4: Club Management, Category Discovery & Membership	Implemented Club entity, ClubRepository, ClubService, ClubController; created ClubCard.jsx, CreateClubModal.jsx.
16–31 Aug	Phase 5: Event Scheduling, Deadlines & Capacity Rules	Implemented ClubEvent entity, ClubEventService, ClubEventController; authored CreateEventModal.jsx, EventCard.jsx.
1–10 Sep	Phase 6: Duplicate Prevention & Exception Handling	Enforced composite uniqueness; coded assertCanManageClub; implemented temporal deadline checks and capacity caps.
11–20 Sep	Phase 7: Dashboard Integration & Cross-Module Hookup	Registered route /clubs in App.jsx; enhanced dashboardService.js with live event aggregation and active club counts.
21–27 Sep	Phase 8: Frontend State Management & Responsive Polish	Authored ClubsPage.jsx with 3-tab switching, category chips, search filters, responsive cards, and loading/error states.
28 Sep–6 Oct	Phase 9: End-to-End Build Verification & Documentation	Executed Maven and Vite builds; verified 15 manual test scenarios; committed code; merged origin/main; prepared project report.

A.2 Individual Git Commit & Verification Log

Below is the exact empirical Git verification log extracted from the project repository for author Akshat Verma:

Verification Field	Verified Repository Value / Empirical Evidence
Developer Name	Akshat Verma
Git Committer Identity	Akshat Verma <vermaakshat0717@gmail.com>
Active Feature Branch	**akshat-feature**

Feature Commit Hash	**77c357d77c04dfc19db3dc793ce546bc6bd0421b (Short: 77c357d)**
Feature Commit Timestamp	Fri Oct 9 2026 21:43:24 +0530
Feature Commit Message	feat(clubs): implement clubs and events module with full backend and UI
Feature Code Statistics	**37 files changed, 3,804 insertions (+), 9 deletions (-)**
Upstream Sync Commit Hash	**244fae83891594a31873d5a778f76f4b2136f672 (Short: 244fae8)**
Upstream Sync Timestamp	Sat Oct 10 2026 22:36:41 +0530
Upstream Merge Message	Merge remote-tracking branch 'origin/main' into akshat-feature
Backend Compilation Result	**Apache Maven 3.9.6 (Java 23.0.2) mvn test-compile: BUILD SUCCESS (2.547 s)**
Frontend Build Result	**Vite 8.3.4 npm run build: SUCCESS (2,209 modules, built in 2.53 s)**
Automated Unit Tests	**Pending verification in branch (Manual verification complete across 15 scenarios)**